

SQL Server 参照完整性实验报告 (Lecture 10)

【23336128】

【梁力航】

实验信息

- **实验名称:** 参照完整性实验
- **实验日期:** 2025-11-13
- **数据库:** School_Data
- **实验目的:** 学习建立外键，以及利用FOREIGN KEY...REFERENCES子句以及各种约束保证参照完整性

实验内容概述

本次实验主要包含以下内容：

1. 不违反参照完整性的插入数据示例
2. 违反参照完整性的插入数据示例
3. 级联删除（CASCADE）
4. ON DELETE NO ACTION 约束
5. ON DELETE SET NULL 约束
6. 表的自参照问题
7. 两张表的互相参照问题

题目1：测试 ON DELETE NO ACTION 约束

实验目的

测试当外键约束设置为 ON DELETE NO ACTION 时，删除父表记录的行为。

实验步骤

1. 创建SC表，初始设置为CASCADE

```
CREATE TABLE SC(  
    sno CHAR(5),  
    cno CHAR(4),  
    grade INT,  
    CONSTRAINT PK_SC PRIMARY KEY (sno,cno),  
    CONSTRAINT FK_SC_Student FOREIGN KEY (sno) REFERENCES Stu_Union(sno) ON DELETE CASCADE,  
    CONSTRAINT FK_SC_Course FOREIGN KEY (cno) REFERENCES Course(cno) ON DELETE CASCADE  
);
```

2. 插入测试数据

```
INSERT INTO SC VALUES ('95002','0001',2);  
INSERT INTO SC VALUES ('95002','0002',2);  
INSERT INTO SC VALUES ('10001','0001',2);  
INSERT INTO SC VALUES ('10001','0002',2);
```

3. 修改外键约束为NO ACTION

```
ALTER TABLE SC DROP CONSTRAINT FK_SC_Student;  
ALTER TABLE SC ADD CONSTRAINT FK_SC_Student  
    FOREIGN KEY (sno) REFERENCES Stu_Union(sno) ON DELETE NO ACTION;
```

4. 尝试删除父表记录

```
DELETE FROM Stu_Union WHERE sno='10001';
```

实验结果

错误信息:

[23000][547] The DELETE statement conflicted with the REFERENCE constraint "FK_SC_St
The conflict occurred in database "School_Data", table "dbo.SC", column 'sno'.

SC表数据（删除失败，数据未变）：

sno	cno	grade

10001	0001	2
10001	0002	2
95002	0001	2
95002	0002	2

结果分析

1. **ON DELETE NO ACTION** 表示当删除父表记录时，如果子表中存在引用该记录的数据，则拒绝删除
2. SC表中存在 sno='10001' 的记录，引用了Stu_Union表中的'10001'
3. 因此删除操作被拒绝，保护了数据的参照完整性
4. 这与CASCADE不同，CASCADE会自动删除子表中的相关记录

题目2：测试 ON DELETE SET NULL 约束

实验目的

测试当外键约束设置为 ON DELETE SET NULL 时，删除父表记录的行为。

实验步骤

1. 创建SC表，使用独立ID作为主键

```
CREATE TABLE SC(  
    sc_id INT IDENTITY(1,1),    -- 使用自增ID作为主键  
    sno CHAR(5) NULL,    -- 允许NULL以支持SET NULL  
    cno CHAR(4) NOT NULL,  
    grade INT,  
    CONSTRAINT PK_SC PRIMARY KEY (sc_id),  
    CONSTRAINT UQ_SC_sno_cno UNIQUE (sno, cno),  
    CONSTRAINT FK_SC_Student FOREIGN KEY (sno) REFERENCES Stu_Union(sno) ON DELETE N  
    CONSTRAINT FK_SC_Course FOREIGN KEY (cno) REFERENCES Course(cno) ON DELETE CASCA  
);
```

2. 插入测试数据

```
INSERT INTO SC (sno, cno, grade) VALUES ('95002','0001',2);
INSERT INTO SC (sno, cno, grade) VALUES ('95002','0002',2);
INSERT INTO SC (sno, cno, grade) VALUES ('10001','0001',2);
INSERT INTO SC (sno, cno, grade) VALUES ('10001','0002',2);
```

3. 修改外键约束为SET NULL

```
ALTER TABLE SC DROP CONSTRAINT FK_SC_Student;
ALTER TABLE SC ADD CONSTRAINT FK_SC_Student
    FOREIGN KEY (sno) REFERENCES Stu_Union(sno) ON DELETE SET NULL;
```

4. 删除父表记录

```
DELETE FROM Stu_Union WHERE sno='10001';
```

实验结果

删除成功！

SC表数据（sno被设置为NULL）：

sno	cno	grade
95002	0001	2
95002	0002	2
NULL	0001	2
NULL	0002	2

结果分析

1. 删除成功，Stu_Union表中 sno='10001' 的记录被删除
2. SC表中原本 sno='10001' 的记录，其sno字段被设置为NULL
3. 这保持了数据的参照完整性，同时保留了选课记录

关键点：

- ON DELETE SET NULL 表示当删除父表记录时，将子表中引用该记录的外键列设置为NULL
- 为了使用SET NULL，外键列必须允许NULL值

- 如果外键列是主键的一部分，则无法使用SET NULL（因为主键不允许NULL）
- 本例中使用独立的ID作为主键，sno作为可空的外键列，因此可以使用SET NULL

题目3：在事务中测试 ON DELETE SET NULL

实验目的

在事务中测试级联操作的链式反

应： students -> Stu_Card (CASCADE) -> ICBC_Card (SET NULL)

实验步骤

1. 修改choices表的外键约束

```
ALTER TABLE choices DROP CONSTRAINT FK_CHOICES_STUDENTS;  
ALTER TABLE choices ADD CONSTRAINT FK_CHOICES_STUDENTS  
    FOREIGN KEY (sid) REFERENCES students(sid) ON DELETE CASCADE;
```

2. 创建Stu_Card和ICBC_Card表

```
CREATE TABLE Stu_Card(  
    card_id CHAR(14),  
    stu_id CHAR(10),  
    remained_money DECIMAL(10,2),  
    CONSTRAINT PK_stu_card PRIMARY KEY (card_id),  
    CONSTRAINT FK_StuCard_Student FOREIGN KEY (stu_id)  
        REFERENCES students(sid) ON DELETE CASCADE  
);
```

```
CREATE TABLE ICBC_Card(  
    bank_id CHAR(20),  
    stu_card_id CHAR(14) NULL,  
    restored_money DECIMAL(10,2),  
    CONSTRAINT PK_Icbc_card PRIMARY KEY (bank_id),  
    CONSTRAINT FK_ICBC_StuCard FOREIGN KEY (stu_card_id)  
        REFERENCES Stu_Card(card_id) ON DELETE CASCADE  
);
```

3. 插入测试数据

```
INSERT INTO Stu_Card VALUES ('05212567','800001216',100.25);
INSERT INTO ICBC_Card VALUES ('9558844022312','05212567',15000.1);
INSERT INTO Stu_Card VALUES ('05212222','800005753',200.50);
INSERT INTO ICBC_Card VALUES ('9558844023645','05212222',50000.3);
```

4. 在事务中修改外键约束并删除数据

```
BEGIN TRANSACTION T3
-- 修改ICBC_Card外键为SET NULL
ALTER TABLE ICBC_Card DROP CONSTRAINT FK_ICBC_StuCard;
ALTER TABLE ICBC_Card ADD CONSTRAINT FK_ICBC_StuCard
    FOREIGN KEY (stu_card_id) REFERENCES Stu_Card(card_id) ON DELETE SET NULL;

-- 删除students表中的记录
DELETE FROM students WHERE sid='800001216';
COMMIT TRANSACTION T3
```

实验结果

初始数据:

Stu_Card表:

card_id	stu_id	remained_money
05212567	800001216	100.25
05212222	800005753	200.50

ICBC_Card表:

bank_id	stu_card_id	restored_money
9558844022312	05212567	15000.1
9558844023645	05212222	50000.3

删除后数据:

Stu_Card表 (card_id='05212567'的记录被级联删除) :

card_id	stu_id	remained_money
05212222	800005753	200.50

ICBC_Card表（stu_card_id被设置为NULL）：

bank_id	stu_card_id	restored_money
9558844022312	NULL	15000.1
9558844023645	05212222	50000.3

结果分析

1. 删除students表中的记录后，由于Stu_Card表设置了 ON DELETE CASCADE ，Stu_Card中对应的记录会被自动删除
2. 当Stu_Card中的记录被删除时，由于ICBC_Card表设置了 ON DELETE SET NULL ，ICBC_Card表中引用该card_id的stu_card_id字段会被设置为NULL
3. 这展示了级联操作的链式反应：
应：students -> Stu_Card (CASCADE) -> ICBC_Card (SET NULL)
4. 事务保证了这一系列操作的原子性

实验结论：

- ON DELETE SET NULL 适用于需要保留子表记录但清除引用关系的场景
- 与CASCADE不同，SET NULL不会删除子表记录，只是将外键列设置为NULL

题目4：表的自参照问题

实验目的

创建一个班里的学生互助表，规定每个学生有且仅有一个帮助对象，帮助对象也必须是班里的学生。

实验步骤

1. 创建学生互助表

```
CREATE TABLE student_help(  
    student_id CHAR(5) NOT NULL,  
    student_name VARCHAR(20),  
    help_target_id CHAR(5) NOT NULL,  
    CONSTRAINT PK_student_help PRIMARY KEY (student_id),  
    CONSTRAINT FK_help_target FOREIGN KEY (help_target_id)  
        REFERENCES student_help(student_id),  
    CONSTRAINT CHK_not_self CHECK (student_id <> help_target_id)  
);
```

2. 插入示例数据（禁用外键约束）

```
ALTER TABLE student_help NOCHECK CONSTRAINT FK_help_target;  
  
INSERT INTO student_help VALUES ('10001', '王勇', '95002');  
INSERT INTO student_help VALUES ('95002', '王敏', '95003');  
INSERT INTO student_help VALUES ('95003', '王洁', '95005');  
INSERT INTO student_help VALUES ('95005', '王杰', '10001');  
  
ALTER TABLE student_help CHECK CONSTRAINT FK_help_target;
```

实验结果

student_help表数据:

学生编号	学生姓名	帮助对象编号	帮助对象姓名
10001	王勇	95002	王敏
95002	王敏	95003	王洁
95003	王洁	95005	王杰
95005	王杰	10001	王勇

结果分析

表结构说明:

- 1. student_id : 学生编号（主键）
- 2. student_name : 学生姓名
- 3. help_target_id : 帮助对象的学号（外键，引用本表的student_id)

4. 约束：每个学生有且仅有一个帮助对象（主键保证唯一性）
5. 约束：帮助对象必须是班里的学生（外键约束）
6. 约束：不能帮助自己（CHECK约束）

自参照特点：

1. 表中的外键引用本表的主键
2. 形成了学生之间的帮助关系网络（循环链表结构）
3. 插入数据时需要注意引用完整性，必须先禁用外键约束，插入所有数据后再启用

题目5：两个表互相参照的问题

实验目的

创建两个互相参照的表，体现部长领导部员（一对多）和部员评测部长（一对一）的关系。

实验步骤

1. 创建部长表（不包含外键）

```
CREATE TABLE department_leaders(  
    leader_id CHAR(5) NOT NULL,  
    leader_name VARCHAR(20),  
    department VARCHAR(30),  
    evaluator_id CHAR(5) NULL,  
    CONSTRAINT PK_leaders PRIMARY KEY (leader_id)  
);
```

2. 创建部员表（包含对部长表的外键）

```
CREATE TABLE department_members(  
    member_id CHAR(5) NOT NULL,  
    member_name VARCHAR(20),  
    department VARCHAR(30),  
    leader_id CHAR(5) NOT NULL,  
    has_evaluation_right BIT DEFAULT 0,  
    CONSTRAINT PK_members PRIMARY KEY (member_id),  
    CONSTRAINT FK_member_leader FOREIGN KEY (leader_id)  
        REFERENCES department_leaders(leader_id) ON DELETE CASCADE  
);
```

3. 为部长表添加外键约束（引用部员表）

```
ALTER TABLE department_leaders  
    ADD CONSTRAINT FK_leader_evaluator FOREIGN KEY (evaluator_id)  
        REFERENCES department_members(member_id) ON DELETE NO ACTION;
```

4. 添加唯一索引约束

```
CREATE UNIQUE INDEX UQ_evaluator_per_dept  
    ON department_members(department, has_evaluation_right)  
    WHERE has_evaluation_right = 1;
```

5. 插入示例数据

```
-- 禁用外键约束
ALTER TABLE department_leaders NOCHECK CONSTRAINT FK_leader_evaluator;

-- 插入部长数据
INSERT INTO department_leaders VALUES ('L0001', '张三', '宣传部', NULL);
INSERT INTO department_leaders VALUES ('L0002', '李四', '组织部', NULL);

-- 插入部员数据
INSERT INTO department_members VALUES ('M0001', '王五', '宣传部', 'L0001', 1);
INSERT INTO department_members VALUES ('M0002', '赵六', '宣传部', 'L0001', 0);
INSERT INTO department_members VALUES ('M0003', '孙七', '组织部', 'L0002', 1);
INSERT INTO department_members VALUES ('M0004', '周八', '组织部', 'L0002', 0);

-- 更新部长表的evaluator_id
UPDATE department_leaders SET evaluator_id = 'M0001' WHERE leader_id = 'L0001';
UPDATE department_leaders SET evaluator_id = 'M0003' WHERE leader_id = 'L0002';

-- 重新启用外键约束
ALTER TABLE department_leaders CHECK CONSTRAINT FK_leader_evaluator;
```

实验结果

领导关系（部长领导的部员）：

部长ID	部长姓名	部门	部员ID	部员姓名	是否有评测权
L0001	张三	宣传部	M0001	王五	是
L0001	张三	宣传部	M0002	赵六	否
L0002	李四	组织部	M0003	孙七	是
L0002	李四	组织部	M0004	周八	否

评测关系（有权评测部长的部员）：

部长ID	部长姓名	部门	评测者ID	评测者姓名
L0001	张三	宣传部	M0001	王五
L0002	李四	组织部	M0003	孙七

结果分析

表结构说明：

1. department_leaders（部长表）：

- leader_id：部长ID（主键）
- leader_name：部长姓名
- department：部门名称
- evaluator_id：有评测权的部员ID（外键 -> department_members）

2. department_members（部员表）：

- member_id：部员ID（主键）
- member_name：部员姓名
- department：部门名称
- leader_id：所属部长ID（外键 -> department_leaders）
- has_evaluation_right：是否有评测权

关系说明：

1. 领导关系：部长(leader_id) <- 部员(leader_id)，一对多
2. 评测关系：部长(evaluator_id) -> 部员(member_id)，一对一
3. 约束：每个部门只有一个部员有评测权（唯一索引）

互参照特点：

1. 两个表互相引用对方的主键作为外键
2. 创建时需要先创建一个表（不含外键），再创建另一个表，最后用ALTER TABLE添加外键
3. 插入数据时需要临时禁用外键约束，或者分步插入并更新
4. 体现了复杂的业务关系：领导关系（一对多）和评测关系（一对一）
5. 由于已经存在CASCADE路径（members -> leaders），反向引用必须使用NO ACTION避免循环级联

实验总结

主要知识点

1. 外键约束的三种删除策略：

- ON DELETE NO ACTION：拒绝删除（默认行为）
- ON DELETE CASCADE：级联删除子表记录
- ON DELETE SET NULL：将子表外键列设置为NULL

2. 参照完整性的重要性：

- 保证数据的一致性和完整性
- 防止出现"孤儿记录"
- 维护表之间的关系约束

3. 特殊场景处理：

- **自参照表**: 表的外键引用自身主键，需要特殊处理插入顺序
- **互参照表**: 两个表互相引用，需要分步创建外键约束
- **循环级联**: 避免多重级联路径，使用NO ACTION打破循环

4. SET NULL的使用条件：

- 外键列必须允许NULL值
- 外键列不能是主键的一部分
- 适用于需要保留子表记录但清除引用关系的场景

注意事项

1. 在设计表关系时，要仔细考虑删除策略的选择
2. 使用SET NULL时，确保外键列允许NULL值
3. 处理互相引用的表时，要注意创建和删除的顺序
4. 避免创建多重级联路径，可能导致意外的数据删除
5. 在生产环境中修改外键约束前，要充分测试影响范围